

Projet Blockchain



By

KHAYAR ANAS

Système de vote décentralisé

ENSIIE

Chapitre 1

Presentation du smart contract

1.1 Introduction

Ce rapport fournit une analyse détaillée du contrat Ballot écrit en Solidity, destiné à être déployé sur la blockchain Ethereum. Le contrat est conçu pour administrer un système de vote électronique sécurisé et transparent.

1.2 Description du Contrat

Le contrat implémente un système de vote où chaque électeur peut soit voter directement pour une proposition, soit déléguer son vote à un autre électeur.

1.2.1 Structures de Données

Voter

La structure `Voter` contient les attributs d'un électeur :

- `weight` : Poids du vote de l'électeur.
- `voted` : Booléen indiquant si l'électeur a déjà voté.
- `delegate` : Adresse de l'électeur à qui le vote est délégué.
- `vote` : Index de la proposition pour laquelle l'électeur a voté.

Proposal

La structure `Proposal` contient les informations d'une proposition :

- `name` : Nom de la proposition.

— `voteCount` : Nombre de votes que la proposition a reçus.

1.2.2 Variables et Modificateur

Owner

L'adresse `owner` représente l'administrateur du contrat, initialement défini au déploiement.

Modifier `onlyOwner`

Ce modificateur est utilisé pour restreindre l'accès de certaines fonctions au seul propriétaire du contrat.

```
1 modifier onlyOwner() {  
2     require(msg.sender == owner, "Only owner can call this function.");  
3     _;  
4 }
```

1.2.3 Fonctions Principales

Constructor

Le constructeur initialise le propriétaire et les propositions initiales.

```
1 constructor() {  
2     owner = msg.sender;  
3     voters[owner].weight = 1;  
4     proposals.push(Proposal({ name: "Proposal 1", voteCount: 0 }));  
5     proposals.push(Proposal({ name: "Proposal 2", voteCount: 0 }));  
6 }
```

`giveRightToVote`

Cette fonction accorde le droit de vote à un électeur, à condition qu'il n'ait pas déjà voté ou reçu le droit de vote.

```
1 function giveRightToVote(address voter) external onlyOwner {  
2     require(!voters[voter].voted, "The voter already voted.");  
3     require(voters[voter].weight == 0, "The voter already has the right  
4         to vote.");  
5     voters[voter].weight = 1;  
6 }
```

delegate

Permet à un électeur de déléguer son vote à un autre électeur.

```
1 function delegate(address to) external {
2     Voter storage sender = voters[msg.sender];
3     require(sender.weight != 0, "You have no right to vote.");
4     require(!sender.voted, "You already voted.");
5     require(to != msg.sender, "Self-delegation is disallowed.");
6
7     while (voters[to].delegate != address(ambassador)) {
8         to = voters[to].delegate;
9         require(to != msg.sender, "Found loop in delegation.");
10    }
11
12    Voter storage delegate_ = voters[to];
13    require(delegate_.weight >= 1);
14
15    sender.voted = true;
16    sender.delegate = to;
17
18    if (delegate_.voted) {
19        proposals[delegate_.vote].voteCount += sender.weight;
20    } else {
21        delegate_.weight += sender.weight;
22    }
23 }
```

vote

Fonction permettant à un électeur de voter pour une proposition spécifique.

```
1 function vote(uint256 proposal) external {
2     Voter storage sender = voters[msg.sender];
3     require(sender.weight != 0, "Has no right to vote.");
4     require(proposal < proposals.length, "Invalid proposal index.");
5     require(!sender.voted, "Already voted.");
6     sender.voted = true;
7     sender.vote = proposal;
8
9     proposals[proposal].voteCount += sender.weight;
```

```
10 }
```

winningProposal et winnerName

Fonctions permettant de déterminer la proposition gagnante et d'obtenir le nom de cette proposition.

```
1 function winningProposal() public view returns (uint256 winningProposal_)
2 {
3     uint256 winningVoteCount = 0;
4     bool isTie = false;
5
6     for (uint256 p = 0; p < proposals.length; p++) {
7         if (proposals[p].voteCount > winningVoteCount) {
8             winningVoteCount = proposals[p].voteCount;
9             winningProposal_ = p;
10            isTie = false;
11        } else if (proposals[p].voteCount == winningVoteCount) {
12            isTie = true;
13        }
14    }
15
16    if (isTie) {
17        winningProposal_ = type(uint256).max; // Special index in case of
18        a tie
19    }
20 }
21
22 function winnerName() external view returns (string memory winnerName_) {
23     uint256 winnerIndex = winningProposal();
24     if (winnerIndex == type(uint256).max) {
25         winnerName_ = "Tie";
26     } else {
27         winnerName_ = proposals[winnerIndex].name;
28     }
29 }
```

Chapitre 2

Améliorations

2.0.1 Problématique de Centralisation

Comme abordé lors de la soutenance, la présence d'un administrateur unique dans le contrat Ballot va à l'encontre du principe de décentralisation inhérent à la blockchain. L'existence d'un *owner* qui contrôle les droits de vote présente un risque potentiel de manipulation des résultats et concentre un pouvoir excessif dans les mains d'une seule entité.

Implications de la Centralisation

La centralisation peut mener à plusieurs problèmes dans un système qui se veut transparent et équitable :

- **Risque de Censure** : L'administrateur peut refuser de donner le droit de vote à certains participants ou de manière arbitraire influencer les résultats.
- **Point de Défaillance Unique** : Si la clé privée de l'administrateur est compromise, tout le système de vote pourrait être mis en péril.
- **Manque de Confiance** : La confiance dans le système de vote peut être érodée si les utilisateurs perçoivent que le processus n'est pas entièrement équitable ou décentralisé.

Solutions pour Renforcer la Décentralisation

Pour atténuer ces problèmes et renforcer la décentralisation du contrat Ballot, plusieurs stratégies peuvent être envisagées :

- **Gouvernance Démocratique** : Implémenter un système de gouvernance où les décisions clés, comme l'ajout de nouvelles propositions ou la modification des règles du contrat, sont prises par un vote majoritaire des participants actuels.

- **Droits de Vote Automatisés** : Automatiser l'attribution des droits de vote via des règles prédéfinies dans le contrat, éliminant ainsi la nécessité d'une intervention manuelle de l'administrateur.
- **Contrats Transparent et Vérifiable** : S'assurer que toutes les actions de l'administrateur sont traçables et vérifiables publiquement sur la blockchain, offrant ainsi une transparence totale.
- **Utilisation de Mécanismes de Consensus** : Introduire des mécanismes de consensus, comme le Proof of Stake ou le Proof of Authority, pour les décisions importantes concernant la gestion du contrat.

En mettant en œuvre ces solutions, le contrat peut devenir plus aligné avec les principes fondamentaux de la technologie blockchain, tout en conservant ses fonctionnalités et en améliorant sa sécurité et sa fiabilité.

2.1 Conclusion

Le projet de contrat Ballot en Solidity a démontré une approche innovante et techniquement avancée pour gérer un système de vote numérique sur la blockchain Ethereum. Ce contrat permet aux utilisateurs de voter sur différentes propositions ou de déléguer leur droit de vote à d'autres, offrant ainsi une flexibilité significative et une interaction utilisateur enrichie.

Cependant, la présence d'un administrateur unique, en l'occurrence l'*owner*, soulève des préoccupations importantes en matière de centralisation, un point qui va à l'encontre des principes fondamentaux de la blockchain. Pour remédier à ces défis, nous avons proposé plusieurs améliorations visant à renforcer la décentralisation du contrat.